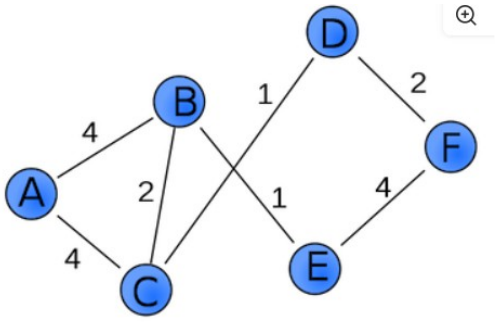
Artificial Intelligence Lab

Dibyasundar Das

20 August 2026

LAB 04



- Translate the visual graph above into Python code using two different data structures. Assume the nodes are labeled A, B, C, D, E, F.
  1. **Adjacency List (using a Dictionary)** Create a variable named `adj_list` where each key is a node and the value is a list of all nodes it is directly connected to.
  2. **Adjacency Matrix (using a Nested List)** Create a variable named `adj_matrix` which is a 2D list (list of lists) representing the connections. Use `1` for a connection and `0` for no connection.
  3. Write python code to Plot graphs in both cases.

> ⚠ **Note**
>
> You must pick a sentinel `math.inf` for "no edge." Explain why `0` is wrong here.
> `

- Heuristic values. Best First Search and A* need an estimate h(n) of the remaining cost from node n to the goal F. Create a dictionary named h from the table below.`

Node	A	B	C	D	E	F
h(n)	6	2	3	2	2	0

- Implement two searches from scratch. Each should return `(path, total_cost, visit_order)`.
  1. `best_first_path(adj_list, h, 'A', 'F')` **Considering the given heuristic only**

2. `astar_path(adj_list, h, 'A', 'F')` **Considering the given heuristic and weights**

- Print one table: with algorithm name, path, edge count , total cost.
- Visualise and compare two figures, showing returned path per algorithm in red.
- Change the heuristic to over-estimation and observe the difference in solution.